Apache Kafka

Lesson 7 · topics, partitions & consumer groups for microservices

In this lesson → ① What is Kafka ② Topics, partitions, offsets ③ Keys & ordering ④ Replication ⑤ Consumer groups ⑥ Scaling rule ⑦ Retention & replay ⑧ Reliable delivery ⑨ Interview Q&A

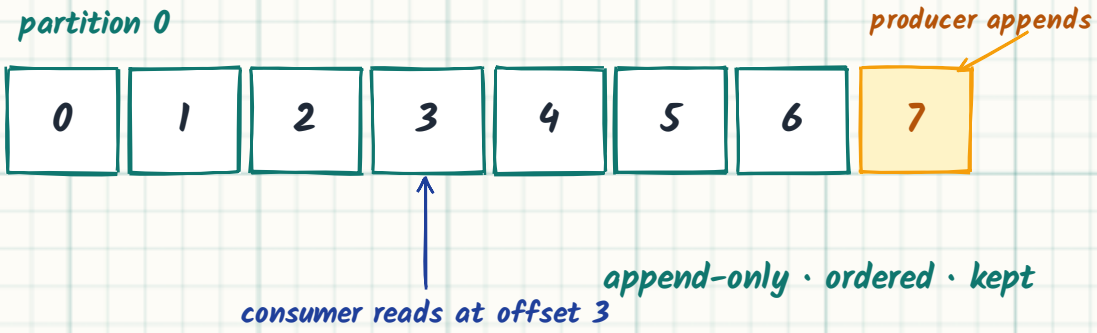
Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① What is Apache Kafka?

> **Kafka** = a distributed **event streaming platform**, built at LinkedIn and open-sourced in 2011. At its core it is a replicated, append-only log stored on disk.

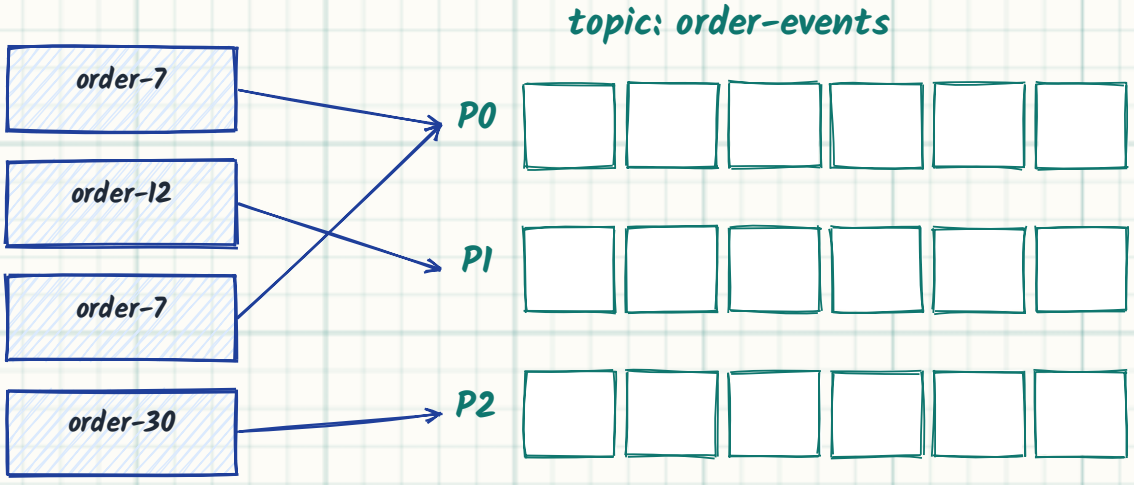
Analogy
A **shared notebook**: producers write new lines at the end; every service reads at its own pace; reading does **not** erase anything.

② Topics, partitions & offsets



Term	Meaning	ShopNest
Topic	named stream of events	order-events
Partition	ordered, append-only log inside a topic	3 partitions: P0, P1, P2
Offset	position of an event in a partition	Notification is at offset 8
Broker	a Kafka server	3 brokers in the cluster

③ Keys & ordering



same key → same partition → in order

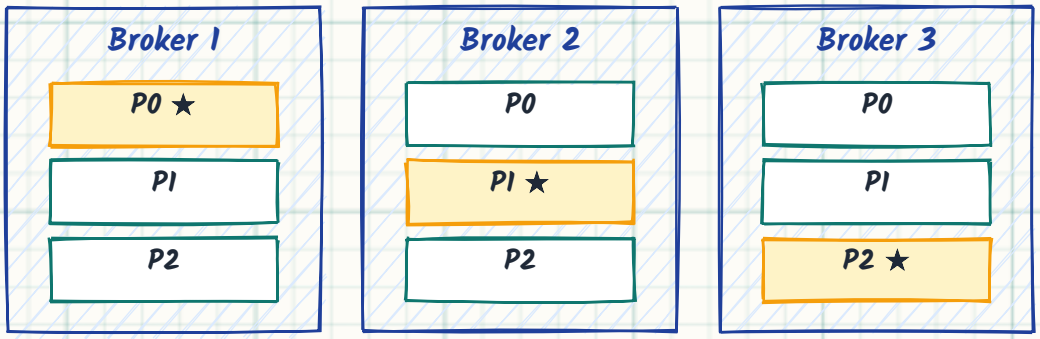
→ Producer sends (key, event) → Kafka hashes the key → picks a partition

→ Same key → **same partition** → events stay **in order**

→ Different keys → spread across partitions → **parallel**

> **Rule**
> Kafka guarantees order **only within a partition**. Key by the thing that must stay in order (e.g. orderId).

④ Brokers & replication



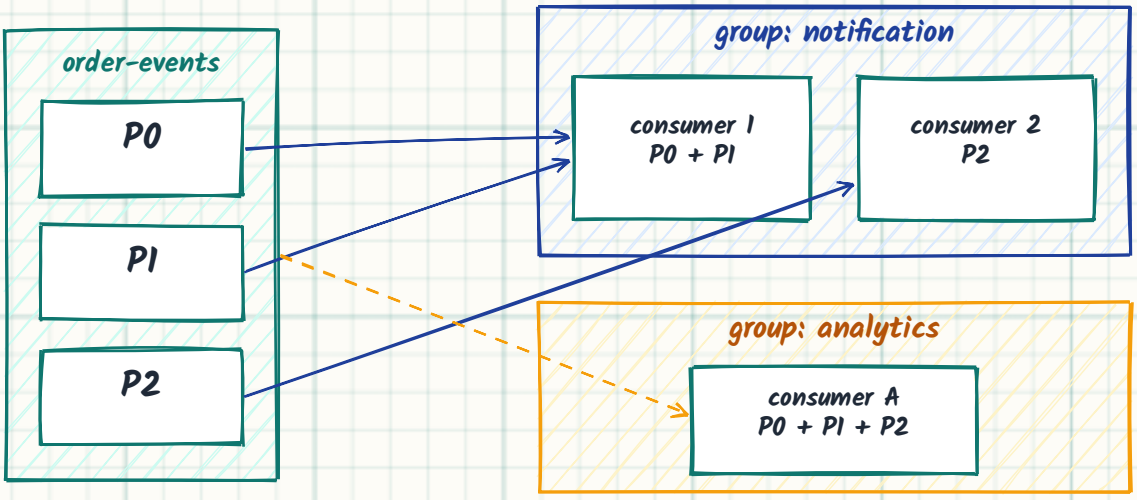
★ leader · others = follower copies (replication factor 3)

→ **Replication factor 3**: each partition copied to 3 brokers

→ **Leader** handles reads/writes; **followers** stay in sync (ISR)

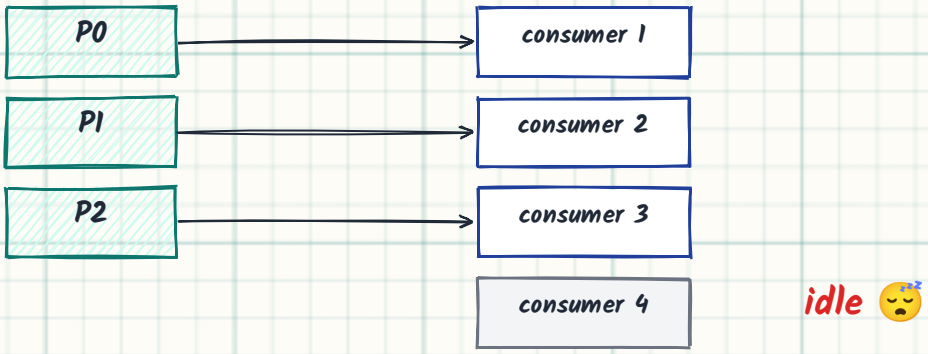
→ Broker dies → a follower becomes leader. Cluster metadata managed by **KRaft** (replaces ZooKeeper)

5 Consumer groups



- ✓ Inside a group, each partition → **exactly one** consumer
- ✓ Add instances to a group → **scale** a service
- ✓ Different groups are **independent**, each with its own offsets

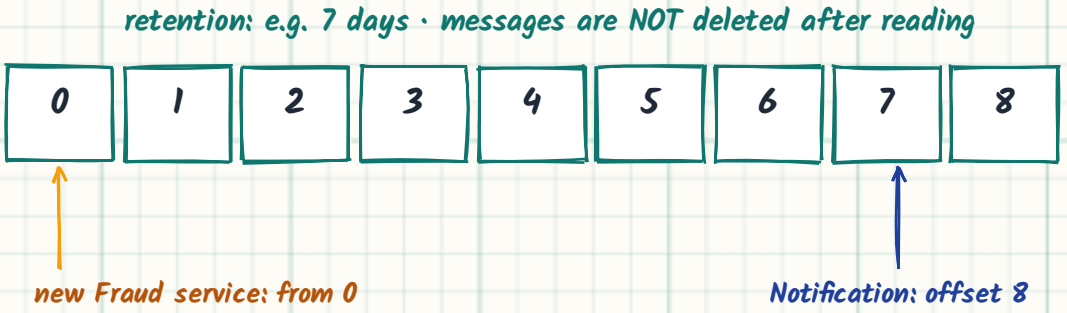
6 The scaling rule



max parallelism = number of partitions

> *Remember!*
> Max parallelism of a group = **number of partitions**. Plan partitions for your peak load.

7 Retention & replay



→ Messages are kept for a **retention period** (time or size), not deleted after reading

→ Each group stores its **own offset**

→ A new service can **replay** history from offset 0

8 Reliable delivery

```
# application.yml
spring.kafka:
  producer:
    acks: all              # wait for all in-sync replicas
    properties.enable.idempotence: true
  consumer:
    group-id: notification
    enable-auto-commit: false  # commit AFTER processing

// key = orderId → all events of one order stay in order
kafkaTemplate.send("order-events", orderId, event);
```

Setting	Why
acks=all	confirm only when all in-sync replicas have the event
Idempotent producer	producer retries don't create duplicates
Commit after processing	crash → re-read → **at-least-once** (keep consumers idempotent)
Key = orderId	events of one order stay in order

9 Kafka vs RabbitMQ

	Kafka	RabbitMQ
Model	distributed log	message queue broker
After read	kept (retention)	removed after ack
Replay	yes, from any offset	not the goal
Ordering	per partition	per queue
Scaling consumers	by partitions	competing consumers
Best for	high-volume event streams, many consumer groups	flexible routing, task queues

10 ShopNest setup

→ Topic `order-events`, 3 partitions, replication factor 3, key = `orderId`

→ Group **notification**: 2 instances · Group **analytics**: 1 instance · Group **stock**: 3 instances

→ New **fraud** service replays the last 7 days on its first start

11 When Kafka is overkill

✗ Low message volume, a few services → a simple queue (RabbitMQ, SQS) is enough

✗ Complex routing / per-message priorities → RabbitMQ fits better

✗ No one to operate a cluster → use a managed service (e.g. Confluent Cloud, Amazon MSK)

Tip
Too few partitions limits scaling later; too many adds overhead. Start with what your peak consumer count needs.

12 Interview Corner

Q. What is a Kafka partition?

A. An ordered, append-only log inside a topic; the unit of ordering and parallelism.

Q. How does Kafka guarantee ordering?

A. Only within a partition. Use a message key (e.g. orderId) so related events go to the same partition.

Q. What is a consumer group?

A. Consumers sharing a topic's partitions, each partition read by one member. Different groups read independently.

Q. What happens if you have more consumers than partitions?

A. Extra consumers sit idle; parallelism = number of partitions.

Q. Kafka vs a traditional queue?

A. Kafka keeps messages after reading (retention), so many groups can read and replay; queues delete after ack.

⚡ Revise in 60 seconds

- → Kafka = replicated **log**
- → Topic → **partitions** → offsets
- → Key → same partition → **order**
- → Replication factor 3 · leader/follower
- → **Consumer groups** share partitions
- → Max parallelism = # partitions
- → Retention → **replay**
- → acks=all · idempotent · commit after

Next lesson → MS-08
The **API Gateway** pattern and Backend-for-Frontend (BFF).
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**